JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class DecimalFormat

java.lang.Object
    java.text.Format
        java.text.NumberFormat
            java.text.DecimalFormat

All Implemented Interfaces:

`Serializable`, `Cloneable`

```
public class DecimalFormat
extends NumberFormat
```

`DecimalFormat` is a concrete subclass of `NumberFormat` that formats decimal numbers in a localized manner. It has a variety of features designed to make it possible to parse and format numbers in any locale, including support for Western, Arabic, and Indic digits. It also supports different kinds of numbers, including integers (123), fixed-point numbers (123.4), scientific notation (1.23E4), percentages (12%), and currency amounts ($123).

Getting a DecimalFormat

To obtain a standard decimal format for a specific locale, including the default locale, it is recommended to call one of the `NumberFormat` `factory methods`, such as `NumberFormat.getInstance()`. These factory methods may not always return a `DecimalFormat` depending on the locale-service provider implementation installed. Thus, to use an instance method defined by `DecimalFormat`, the `NumberFormat` returned by the factory method should be type checked before converted to `DecimalFormat`. If the installed locale-sensitive service implementation does not support the given `Locale`, the parent locale chain will be looked up, and a `Locale` used that is supported.

If the factory methods are not desired, use one of the constructors such as `DecimalFormat(String pattern)`. See the `Pattern` section for more information on the `pattern` parameter.

Using DecimalFormat

The following is an example of formatting and parsing,

```
NumberFormat nFmt = NumberFormat.getCurrencyInstance(Locale.US);
if (nFmt instanceof DecimalFormat dFmt) {
    // pattern match to DecimalFormat to use setPositiveSuffix(String)
    dFmt.setPositiveSuffix(" dollars");
    dFmt.format(100000); // returns "$100,000.00 dollars"
```

```
    dFmt.parse("$100,000.00 dollars"); // returns 100000
}
```

Formatting and Parsing

Rounding

When formatting, `DecimalFormat` can adjust its rounding using `setRoundingMode(RoundingMode)`. By default, it uses `RoundingMode.HALF_EVEN`.

Digits

When formatting, `DecimalFormat` uses the ten consecutive characters starting with the localized zero digit defined in the `DecimalFormatSymbols` object as digits.

When parsing, these digits as well as all Unicode decimal digits, as defined by `Character.digit`, are recognized.

Integer and Fraction Digit Limits

The integer and fraction digit limits are set by either applying a `pattern` or using one of the appropriate `DecimalFormat` setter methods, for example, `setMinimumFractionDigits(int)`. These limits have no impact on parsing behavior.

Special Values

- **Not a Number** (`NaN`) is formatted as a string, which is typically given as "NaN". This string is determined by `DecimalFormatSymbols.getNaN()`. This
  is the only value for which the prefixes and suffixes are not attached.
- **Infinity** is formatted as a string, which is typically given as `"∞"` (`U+221E`), with the positive or negative prefixes and suffixes attached. This string is determined by `DecimalFormatSymbols.getInfinity()`.
- **Negative zero** (`"-0"`) parses to
  - `BigDecimal(0)` if `isParseBigDecimal()` is true
  - `Long(0)` if `isParseBigDecimal()` is false and `isParseIntegerOnly()` is true
  - `Double(-0.0)` if both `isParseBigDecimal()` and `isParseIntegerOnly()` are false

Synchronization

Decimal formats are generally not synchronized. It is recommended to create separate format instances for each thread. If multiple threads access a format concurrently, it must be synchronized externally.

DecimalFormat Pattern

A `DecimalFormat` comprises a *pattern* and a set of *symbols*. The pattern may be set directly using `applyPattern()`, or indirectly using the various API methods. The symbols are stored in a `DecimalFormatSymbols` object. When using the `NumberFormat` factory methods, the pattern and symbols are created from the locale-sensitive service implementation installed.

`DecimalFormat` patterns have the following syntax:

```
Pattern:
        PositivePattern
        PositivePattern ; NegativePattern
PositivePattern:
        Prefix_opt Number Suffix_opt
NegativePattern:
        Prefix_opt Number Suffix_opt
Prefix:
        Any characters except the special pattern characters
Suffix:
        Any characters except the special pattern characters
Number:
        Integer Exponent_opt
        Integer . Fraction Exponent_opt
Integer:
        MinimumInteger
        #
        # Integer
        # , Integer
MinimumInteger:
        0
        0 MinimumInteger
        0 , MinimumInteger
Fraction:
        MinimumFraction_opt OptionalFraction_opt
MinimumFraction:
        0 MinimumFraction_opt
```

```
OptionalFraction:
        # OptionalFraction_opt
Exponent:
        E MinimumExponent
MinimumExponent:
        0 MinimumExponent_opt
```

Special Pattern Characters

The special characters in the table below are interpreted syntactically when used in the DecimalFormat pattern. They must be quoted, unless noted otherwise, if they are to appear in the prefix or suffix as literals.

The characters in the `Symbol` column are used in non-localized patterns. The corresponding characters in the `Localized Symbol` column are used in localized patterns, with the characters in `Symbol` losing their syntactical meaning. Two exceptions are the currency sign (`U+00A4`) and quote (`U+0027`), which are not localized.

Non-localized patterns should be used when calling `applyPattern(String)`. Localized patterns should be used when calling `applyLocalizedPattern(String)`.

Symbol	Localized Symbol	Location	Meaning
`0`	`DecimalFormatSymbols.getZeroDigit()`	Number	Digit
`#`	`DecimalFormatSymbols.getDigit()`	Number	Digit, zero shows as absent
`.`	`DecimalFormatSymbols.getDecimalSeparator()`	Number	Decimal separator monetary decimal separator
`- (U+002D)`	`DecimalFormatSymbols.getMinusSign()`	Number	Minus sign
`,`	`DecimalFormatSymbols.getGroupingSeparator()`	Number	Grouping separator monetary grouping separator
`E`	`DecimalFormatSymbols.getExponentSeparator()`	Number	Separates mantissa and exponent in scientific

Symbol	Localized Symbol	Location	Meaning
			notation. This value case sensistive. *Need not b quoted in prefix or suffix.*
;	DecimalFormatSymbols.getPatternSeparator()	Subpattern boundary	Separates positive an negative subpattern
%	DecimalFormatSymbols.getPercent()	Prefix or suffix	Multiply by 100 and show as percentage
‰ (U+2030)	DecimalFormatSymbols.getPerMill()	Prefix or suffix	Multiply by 1000 and show as pe mille value
¤ (U+00A4)	n/a (not localized)	Prefix or suffix	Currency sign, replaced b currency symbol. If doubled, replaced b internation currency symbol. If present in pattern, th monetary decimal/ grouping separators are used instead of the decima grouping separators
' (U+0027)	n/a (not localized)		

Symbol	Localized Symbol	Location	Meaning
		Prefix or suffix	Used to quote special characters in a prefix or suffix, for example, `"'#'#"` formats 123 to `"#123"`. To create a single quote itself, use two in a row: `"# o''clock"`

Maximum Digits Derivation

For any given `DecimalFormat` pattern, if the pattern is not in scientific notation, the maximum number of integer digits will not be derived from the pattern, and instead set to `Integer.MAX_VALUE`. Otherwise, if the pattern is in scientific notation, the maximum number of integer digits will be derived from the pattern. This derivation is detailed in the `Scientific Notation` section. `setMaximumIntegerDigits(int)` can be used to manually adjust the maximum integer digits.

Negative Subpatterns

A `DecimalFormat` pattern contains a positive and negative subpattern, for example, `"#,##0.00;(#,##0.00)"`. Each subpattern has a prefix, numeric part, and suffix. The negative subpattern is optional; if absent, then the positive subpattern prefixed with the minus sign `'-' (U+002D HYPHEN-MINUS)` is used as the negative subpattern. That is, `"0.00"` alone is equivalent to `"0.00;-0.00"`. If there is an explicit negative subpattern, it serves only to specify the negative prefix and suffix; the number of digits, minimal digits, and other characteristics are all the same as the positive pattern. That means that `"#,##0.0#;(#)"` produces precisely the same behavior as `"#,##0.0#;(#,##0.0#)"`. In `lenient parsing` mode, loose matching of the minus sign pattern is enabled, following the LDML's loose matching specification.

The prefixes, suffixes, and various symbols used for infinity, digits, grouping separators, decimal separators, etc. may be set to arbitrary values, and they will appear properly during formatting. However, care must be taken that the

symbols and strings do not conflict, or parsing will be unreliable. For example, either the positive and negative prefixes or the suffixes must be distinct for `DecimalFormat.parse()` to be able to distinguish positive from negative values. (If they are identical, then `DecimalFormat` will behave as if no negative subpattern was specified.) Another example is that the decimal separator and grouping
separator should be distinct characters, or parsing will be impossible.

Grouping Separator

The grouping separator is commonly used for thousands, but in some locales it separates ten-thousands. The grouping size is a constant number of digits between the grouping characters, such as 3 for 100,000,000 or 4 for 1,0000,0000. If you supply a pattern with multiple grouping characters, the interval between the last one and the end of the integer is the one that is used. For example, `"#,##,###,####" == "######,####" == "##,####,####"`.

Scientific Notation

Numbers in scientific notation are expressed as the product of a mantissa and a power of ten, for example, 1234 can be expressed as 1.234 x 10^3. The mantissa is often in the range 1.0 ≤ x < 10.0, but it need not be. `DecimalFormat` can be instructed to format and parse scientific notation *only via a pattern*; there is currently no factory method that creates a scientific notation format. In a pattern, the exponent character immediately followed by one or more digit characters indicates scientific notation. Example: `"0.###E0"` formats the number 1234 as `"1.234E3"`.

- The number of digit characters after the exponent character gives the minimum exponent digit count. There is no maximum. Negative exponents are formatted using the localized minus sign, *not* the prefix and suffix from the pattern. This allows patterns such as `"0.###E0 m/s"`.
- The *maximum integer* digits is the sum of '0's and '#'s prior to the decimal point. The *minimum integer* digits is the sum of the '0's prior to the decimal point. The *maximum fraction* and *minimum fraction* digits follow the same rules, but apply to the digits after the decimal point but before the exponent. For example, the following pattern: `"#00.0####E0"` would have a minimum number of integer digits = 2("00") and a maximum number of integer digits = 3("#00"). It would have a minimum number of fraction digits = 1("0") and a maximum number of fraction digits= 5("0####").
- The minimum and maximum number of integer digits are interpreted together:
    - If the maximum number of integer digits is greater than their minimum number and greater than 1, it forces the exponent to be a multiple of the maximum number of integer digits, and the minimum

number of integer digits to be interpreted as 1. The most common use of this is to generate *engineering notation*, in which the exponent is a multiple of three, e.g., `"##0.#####E0"`. Using this pattern, the number 12345 formats to `"12.345E3"`, and 123456 formats to `"123.456E3"`.
    - Otherwise, the minimum number of integer digits is achieved by adjusting the exponent. Example: 0.00123 formatted with `"00.###E0"` yields `"12.3E-4"`.
- For a given number, the amount of significant digits in the mantissa can be calculated as such

```
Mantissa Digits:
        min(max(Minimum Pattern Digits, Original Number Digits), Maximum
Minimum pattern Digits:
        Minimum Integer Digits + Minimum Fraction Digits
Maximum pattern Digits:
        Maximum Integer Digits + Maximum Fraction Digits
Original Number Digits:
        The amount of significant digits in the number to be formatted
```

  This means that generally, a mantissa will have up to the combined maximum integer and fraction digits, if the original number itself has enough significant digits. However, if there are more minimum pattern digits than significant digits in the original number, the mantissa will have significant digits that equals the combined minimum integer and fraction digits. The number of significant digits does not affect parsing.

  It should be noted, that the integer portion of the mantissa will give any excess digits to the fraction portion, whether it be for precision or for satisfying the total amount of combined minimum digits.

  This behavior can be observed in the following example,

```
DecimalFormat df = new DecimalFormat("#000.000##E0");
df.format(12); // returns "12.0000E0"
df.format(123456789) // returns "1.23456789E8"
```

- Exponential patterns may not contain grouping separators.

Implementation Requirements:

When formatting a `Number` other than `BigInteger` and `BigDecimal`, `309` is used as the upper limit for integer digits, and `340` as the upper limit for fraction digits. This occurs, even if one of the `DecimalFormat` getter methods,

for example, `getMinimumFractionDigits()` returns a numerically greater value.

Since:

1.1

External Specifications

Unicode Locale Data Markup Language (LDML)

See Also:

Java Tutorial, `NumberFormat`, `DecimalFormatSymbols`, `ParsePosition`, `Locale`, Serialized Form

Nested Class Summary

Nested classes/interfaces declared in class NumberFormat
`NumberFormat.Field`, `NumberFormat.Style`

Field Summary

Fields declared in class NumberFormat
`FRACTION_FIELD`, `INTEGER_FIELD`

Constructor Summary

Constructors

Constructor	Description
`DecimalFormat()`	Creates a DecimalFormat using the default pattern and symbols for the default `FORMAT` locale.
`DecimalFormat(String pattern)`	Creates a DecimalFormat using the given pattern and the symbols for the default `FORMAT` locale.
`DecimalFormat(String pattern, DecimalFormatSymbols symbols)`	Creates a DecimalFormat using the given pattern and symbols.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
void	**applyLocalizedPattern** (String pattern)	Apply the given pattern to this Format object.
void	**applyPattern** (String pattern)	Apply the given pattern to this Format object.
Object	**clone**()	Standard override; no change in semantics.
boolean	**equals**(Object obj)	Compares the specified object with this DecimalFormat for equality.
StringBuffer	**format**(double number, StringBuffer result, FieldPosition fieldPosit	Formats a double to produce a string.
StringBuffer	**format**(long number, StringBuffer result, FieldPosition fieldPosit	Format a long to produce a string.
final StringBuffer	**format**(Object number, StringBuffer toAppendTo, FieldPosition pos)	Formats a number and appends the resulting text to the given string buffer.
AttributedCharacterI	**formatToCharacterIterato** (Object obj)	Formats an Object producing an AttributedCharacterIterator.
Currency	**getCurrency**()	Gets the currency used by this decimal format when formatting currency values.
DecimalFormatSymbols	**getDecimalFormatSymbols**(	Returns a copy of the decimal format symbols, which is generally not changed by the programmer or user.
int	**getGroupingSize**()	Return the grouping size.
int	**getMaximumFractionDigits**	

		Returns the maximum number of digits allowed in the fraction portion of a number during formatting.
int	**getMaximumIntegerDigits**(	Returns the maximum number of digits allowed in the integer portion of a number during formatting.
int	**getMinimumFractionDigits**	Returns the minimum number of digits allowed in the fraction portion of a number during formatting.
int	**getMinimumIntegerDigits**(	Returns the minimum number of digits allowed in the integer portion of a number during formatting.
int	**getMultiplier**()	Gets the multiplier for use in percent, per mille, and similar formats.
String	**getNegativePrefix**()	Get the negative prefix.
String	**getNegativeSuffix**()	Get the negative suffix.
String	**getPositivePrefix**()	Get the positive prefix.
String	**getPositiveSuffix**()	Get the positive suffix.
RoundingMode	**getRoundingMode**()	Gets the RoundingMode used in this DecimalFormat.
int	**hashCode**()	Returns the hash code for this DecimalFormat.
boolean	**isDecimalSeparatorAlways**	Allows you to get the behavior of the decimal separator with integers.

boolean	**isParseBigDecimal**()	Returns whether the parse(java.lang.String, java.text.ParsePosition) method returns BigDecimal.
boolean	**isStrict**()	Returns true if this format will parse numbers strictly; false otherwise.
Number	**parse**(String text, ParsePosition pos)	Parses text from the beginning of the given string to produce a Number.
void	**setCurrency** (Currency currency)	Sets the currency used by this number format when formatting currency values.
void	**setDecimalFormatSymbols** (DecimalFormatSymbols ne	Sets the decimal format symbols, which is generally not changed by the programmer or user.
void	**setDecimalSeparatorAlway** (boolean newValue)	Allows you to set the behavior of the decimal separator with integers.
void	**setGroupingSize** (int newValue)	Set the grouping size.
void	**setMaximumFractionDigits** (int newValue)	Sets the maximum number of digits allowed in the fraction portion of a number during formatting.
void	**setMaximumIntegerDigits** (int newValue)	Sets the maximum number of digits allowed in the integer portion of a number during formatting.

void	**setMinimumFractionDigits**(int newValue)	Sets the minimum number of digits allowed in the fraction portion of a number during formatting.
void	**setMinimumIntegerDigits**(int newValue)	Sets the minimum number of digits allowed in the integer portion of a number during formatting.
void	**setMultiplier**(int newValue)	Sets the multiplier for use in percent, per mille, and similar formats.
void	**setNegativePrefix**(String newValue)	Set the negative prefix.
void	**setNegativeSuffix**(String newValue)	Set the negative suffix.
void	**setParseBigDecimal**(boolean newValue)	Sets whether the parse(java.lang.String, java.text.ParsePosition) method returns BigDecimal.
void	**setPositivePrefix**(String newValue)	Set the positive prefix.
void	**setPositiveSuffix**(String newValue)	Set the positive suffix.
void	**setRoundingMode**(RoundingMode roundingMo	Sets the RoundingMode used in this DecimalFormat.
void	**setStrict**(boolean strict)	Change the leniency value for parsing.
String	**toLocalizedPattern**()	Synthesizes a localized pattern string that represents the current state of this Format object.

String	**toPattern**()	Synthesizes a pattern string that represents the current state of this Format object.
String	**toString**()	Returns a string identifying this DecimalFormat, for debugging.

Methods declared in class NumberFormat

format, format, getAvailableLocales, getCompactNumberInstance, getCompactNumberInstance, getCurrencyInstance, getCurrencyInstance, getInstance, getInstance, getIntegerInstance, getIntegerInstance, getNumberInstance, getNumberInstance, getPercentInstance, getPercentInstance, isGroupingUsed, isParseIntegerOnly, parse, parseObject, setGroupingUsed, setParseIntegerOnly

Methods declared in class Format

format, parseObject

Methods declared in class Object

finalize, getClass, notify, notifyAll, wait, wait, wait

Constructor Details

DecimalFormat

```
public DecimalFormat()
```

Creates a DecimalFormat using the default pattern and symbols for the default FORMAT locale. This is a convenient way to obtain a DecimalFormat when internationalization is not the main concern.

API Note:

To obtain standard formats for a given locale, use the NumberFormat factory methods such as NumberFormat.getNumberInstance(Locale). These factories will return the most appropriate subclass of NumberFormat for a given locale.

See Also:

NumberFormat.getInstance(Locale),

NumberFormat.getNumberInstance(Locale),
NumberFormat.getCurrencyInstance(Locale),
NumberFormat.getPercentInstance(Locale)

DecimalFormat

```
public DecimalFormat(String pattern)
```

Creates a DecimalFormat using the given pattern and the symbols for the default FORMAT locale. This is a convenient way to obtain a DecimalFormat when internationalization is not the main concern. The number of maximum integer digits is usually not derived from the pattern. See the note in the Patterns section for more detail.

API Note:

To obtain standard formats for a given locale, use the NumberFormat factory methods such as NumberFormat.getNumberInstance(Locale). These factories will return the most appropriate subclass of NumberFormat for a given locale.

Parameters:

pattern - a non-localized pattern string.

Throws:

NullPointerException - if pattern is null

IllegalArgumentException - if the given pattern is invalid.

See Also:

NumberFormat.getInstance(Locale),
NumberFormat.getNumberInstance(Locale),
NumberFormat.getCurrencyInstance(Locale),
NumberFormat.getPercentInstance(Locale)

DecimalFormat

```
public DecimalFormat(String pattern,
                     DecimalFormatSymbols symbols)
```

Creates a DecimalFormat using the given pattern and symbols. Use this constructor when you need to completely customize the behavior of the format. The number of maximum integer digits is usually not derived from the pattern. See the note in the Patterns section for more detail.

API Note:

To obtain standard formats for a given locale, use the NumberFormat factory methods such as NumberFormat.getInstance(Locale) or NumberFormat.getCurrencyInstance(Locale). If you need only minor

adjustments to a standard format, you can modify the format returned by a NumberFormat factory method.

Parameters:

`pattern` - a non-localized pattern string

`symbols` - the set of symbols to be used

Throws:

`NullPointerException` - if any of the given arguments is null

`IllegalArgumentException` - if the given pattern is invalid

See Also:

`NumberFormat.getInstance(Locale)`,
`NumberFormat.getNumberInstance(Locale)`,
`NumberFormat.getCurrencyInstance(Locale)`,
`NumberFormat.getPercentInstance(Locale)`,
`DecimalFormatSymbols`

Method Details

format

```
public final StringBuffer format(Object number,
                                 StringBuffer toAppendTo,
                                 FieldPosition pos)
```

Formats a number and appends the resulting text to the given string buffer. The number can be of any subclass of `Number`.

Overrides:

`format` in class `NumberFormat`

Implementation Requirements:

This implementation uses the maximum precision permitted.

Parameters:

`number` - the number to format

`toAppendTo` - the `StringBuffer` to which the formatted text is to be appended

`pos` - keeps track on the position of the field within the returned string. For example, for formatting a number `1234567.89` in `Locale.US` locale, if the given `fieldPosition` is `NumberFormat.INTEGER_FIELD`, the begin index and end index of `fieldPosition` will be set to 0 and 9, respectively for the output string `1,234,567.89`.

Returns:

the value passed in as `toAppendTo`

Throws:

`IllegalArgumentException` - if `number` is null or not an instance of `Number`.

`NullPointerException` - if `toAppendTo` or `pos` is null

`ArithmeticException` - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

`FieldPosition`

format

```
public StringBuffer format(double number,
                           StringBuffer result,
                           FieldPosition fieldPosition)
```

Formats a double to produce a string.

Specified by:

`format` in class `NumberFormat`

Parameters:

`number` - The double to format

`result` - where the text is to be appended

`fieldPosition` - keeps track on the position of the field within the returned string. For example, for formatting a number `1234567.89` in `Locale.US` locale, if the given `fieldPosition` is `NumberFormat.INTEGER_FIELD`, the begin index and end index of `fieldPosition` will be set to 0 and 9, respectively for the output string `1,234,567.89`.

Returns:

The formatted number string

Throws:

`NullPointerException` - if `result` or `fieldPosition` is `null`

`ArithmeticException` - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

`FieldPosition`

format

```
public StringBuffer format(long number,
                           StringBuffer result,
                           FieldPosition fieldPosition)
```

Format a long to produce a string.

Specified by:

`format` in class `NumberFormat`

Parameters:

`number` - The long to format

`result` - where the text is to be appended

`fieldPosition` - keeps track on the position of the field within the returned string. For example, for formatting a number `123456789` in `Locale.US` locale, if the given `fieldPosition` is `NumberFormat.INTEGER_FIELD`, the begin index and end index of `fieldPosition` will be set to 0 and 11, respectively for the output string `123,456,789`.

Returns:

The formatted number string

Throws:

`NullPointerException` - if `result` or `fieldPosition` is `null`

`ArithmeticException` - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

See Also:

`FieldPosition`

formatToCharacterIterator

```
public AttributedCharacterIterator formatToCharacterIterator
(Object obj)
```

Formats an Object producing an `AttributedCharacterIterator`. You can use the returned `AttributedCharacterIterator` to build the resulting String, as well as to determine information about the resulting String.

Each attribute key of the AttributedCharacterIterator will be of type `NumberFormat.Field`, with the attribute value being the same as the attribute key.

Overrides:

`formatToCharacterIterator` in class `Format`

Parameters:

`obj` - The object to format

Returns:

AttributedCharacterIterator describing the formatted value.

Throws:

`NullPointerException` - if obj is null.

`IllegalArgumentException` - when the Format cannot format the given object.

`ArithmeticException` - if rounding is needed with rounding mode being set to RoundingMode.UNNECESSARY

Since:

1.4

parse

```
public Number parse(String text,
                    ParsePosition pos)
```

Parses text from the beginning of the given string to produce a `Number`.

This method attempts to parse text starting at the index given by the `ParsePosition`. If parsing succeeds, then the index of the `ParsePosition` is updated to the index after the last character used (parsing does not necessarily use all characters up to the end of the string), and the parsed number is returned. The updated `ParsePosition` can be used to indicate the starting point for the next call to this method. If an error occurs, then the index of the `ParsePosition` is not changed, the error index of the `ParsePosition` is set to the index of the character where the error occurred, and `null` is returned.

This method will return a Long if possible (e.g., within the range [Long.MIN_VALUE, Long.MAX_VALUE] and with no decimals), otherwise a Double.

Parsing can be done in either a strict or lenient manner, by default it is lenient.

Parsing fails when **lenient**, if the prefix and/or suffix are non-empty and cannot be found due to parsing ending early, or the first character after the prefix cannot be parsed.

Parsing fails when **strict**, if in `text`,

- The prefix is not found. For example, a `Locale.US` currency format prefix: "$"
- The suffix is not found. For example, a `Locale.US` percent format suffix: "%"
- `NumberFormat.isGroupingUsed()` returns `true`, and `getGroupingSize()` is not adhered to
- `NumberFormat.isGroupingUsed()` returns `false`, and the grouping symbol is found

- NumberFormat.isGroupingUsed() returns true and the grouping symbol occurs after the decimal separator
- Any other characters are found, that are not the expected symbols, and are not digits that occur within the numerical portion

When lenient, the minus sign in the negative subpatterns is loosely matched against lenient minus sign characters.

The subclass returned depends on the value of isParseBigDecimal() as well as on the string being parsed.

- If isParseBigDecimal() is false (the default), most integer values are returned as Long objects, no matter how they are written: "17" and "17.000" both parse to Long(17). Values that cannot fit into a Long are returned as Doubles. This includes values with a fractional part, infinite values, NaN, and the value -0.0. DecimalFormat does *not* decide whether to return a Double or a Long based on the presence of a decimal separator in the source string. Doing so would prevent integers that overflow the mantissa of a double, such as "-9,223,372,036,854,775,808.00", from being parsed accurately.

  Callers may use the Number methods doubleValue, longValue, etc., to obtain the type they want.

- If isParseBigDecimal() is true, values are returned as BigDecimal objects. The values are the ones constructed by BigDecimal(String) for corresponding strings in locale-independent format. The special cases negative and positive infinity and NaN are returned as Double instances holding the values of the corresponding Double constants.

DecimalFormat parses all Unicode characters that represent decimal digits, as defined by Character.digit(). In addition, DecimalFormat also recognizes as digits the ten consecutive characters starting with the localized zero digit defined in the DecimalFormatSymbols object.

Specified by:

parse in class NumberFormat

Parameters:

text - the string to be parsed

pos - A ParsePosition object with index and error index information as described above.

Returns:

the parsed value, or null if the parse fails

Throws:

NullPointerException - if text or pos is null.

See Also:

`NumberFormat.isStrict()`

getDecimalFormatSymbols

`public DecimalFormatSymbols getDecimalFormatSymbols()`

Returns a copy of the decimal format symbols, which is generally not changed by the programmer or user.

Returns:

a copy of the desired DecimalFormatSymbols

See Also:

`DecimalFormatSymbols`

setDecimalFormatSymbols

`public void setDecimalFormatSymbols(DecimalFormatSymbols newSymbols)`

Sets the decimal format symbols, which is generally not changed by the programmer or user.

Parameters:

`newSymbols` - desired DecimalFormatSymbols

See Also:

`DecimalFormatSymbols`

getPositivePrefix

`public String getPositivePrefix()`

Get the positive prefix.

Examples: +123, $123, sFr123

Returns:

the positive prefix

setPositivePrefix

`public void setPositivePrefix(String newValue)`

Set the positive prefix.

Examples: +123, $123, sFr123

Parameters:

`newValue` - the new positive prefix. Non-null.

Throws:

`NullPointerException` - if `newValue` is `null`

getNegativePrefix

```
public String getNegativePrefix()
```

Get the negative prefix.

Examples: -123, ($123) (with negative suffix), sFr-123

Returns:

the negative prefix

setNegativePrefix

```
public void setNegativePrefix(String newValue)
```

Set the negative prefix.

Examples: -123, ($123) (with negative suffix), sFr-123

Parameters:

`newValue` - the new negative prefix. Non-null.

Throws:

`NullPointerException` - if `newValue` is `null`

getPositiveSuffix

```
public String getPositiveSuffix()
```

Get the positive suffix.

Example: 123%

Returns:

the positive suffix

setPositiveSuffix

```
public void setPositiveSuffix(String newValue)
```

Set the positive suffix.

Example: 123%

Parameters:

`newValue` - the new positive suffix. Non-null.

Throws:

NullPointerException - if newValue is null

getNegativeSuffix

```
public String getNegativeSuffix()
```

Get the negative suffix.

Examples: -123%, ($123) (with positive suffixes)

Returns:

the negative suffix

setNegativeSuffix

```
public void setNegativeSuffix(String newValue)
```

Set the negative suffix.

Examples: 123%

Parameters:

newValue - the new negative suffix. Non-null.

Throws:

NullPointerException - if newValue is null

getMultiplier

```
public int getMultiplier()
```

Gets the multiplier for use in percent, per mille, and similar formats.

Returns:

the multiplier

See Also:

setMultiplier(int)

setMultiplier

```
public void setMultiplier(int newValue)
```

Sets the multiplier for use in percent, per mille, and similar formats. For a percent format, set the multiplier to 100 and the suffixes to have '%' (for Arabic, use the Arabic percent sign). For a per mille format, set the multiplier to 1000 and the suffixes to have 'U+2030'.

Example: with multiplier 100, 1.23 is formatted as "123", and "123" is parsed into 1.23. If isParseIntegerOnly() returns true, "123" is parsed into 1.

Parameters:

`newValue` - the new multiplier

See Also:

`getMultiplier()`

getGroupingSize

```
public int getGroupingSize()
```

Return the grouping size. Grouping size is the number of digits between grouping separators in the integer portion of a number. For example, in the number "123,456.78", the grouping size is 3. Grouping size of zero designates that grouping is not used, which provides the same formatting as if calling `setGroupingUsed(false)`.

Returns:

the grouping size

See Also:

`setGroupingSize(int)`,
`NumberFormat.isGroupingUsed()`,
`DecimalFormatSymbols.getGroupingSeparator()`

setGroupingSize

```
public void setGroupingSize(int newValue)
```

Set the grouping size. Grouping size is the number of digits between grouping separators in the integer portion of a number. For example, in the number "123,456.78", the grouping size is 3. Grouping size of zero designates that grouping is not used, which provides the same formatting as if calling `setGroupingUsed(false)`.

The value passed in is converted to a byte, which may lose information. Values that are negative or greater than `Byte.MAX_VALUE`, will throw an `IllegalArgumentException`.

Parameters:

`newValue` - the new grouping size

Throws:

`IllegalArgumentException` - if `newValue` is negative or greater than `Byte.MAX_VALUE`

See Also:

`getGroupingSize()`,
`NumberFormat.setGroupingUsed(boolean)`,
`DecimalFormatSymbols.setGroupingSeparator(char)`

isDecimalSeparatorAlwaysShown

`public boolean isDecimalSeparatorAlwaysShown()`

Allows you to get the behavior of the decimal separator with integers. (The decimal separator will always appear with decimals.)

Example: Decimal ON: 12345 → 12345.; OFF: 12345 → 12345

Returns:

`true` if the decimal separator is always shown; `false` otherwise

setDecimalSeparatorAlwaysShown

`public void setDecimalSeparatorAlwaysShown(boolean newValue)`

Allows you to set the behavior of the decimal separator with integers. (The decimal separator will always appear with decimals.)

Example: Decimal ON: 12345 → 12345.; OFF: 12345 → 12345

Parameters:

`newValue` - `true` if the decimal separator is always shown; `false` otherwise

isStrict

`public boolean isStrict()`

Returns `true` if this format will parse numbers strictly; `false` otherwise.

Overrides:

`isStrict` in class `NumberFormat`

Returns:

`true` if this format will parse numbers strictly; `false` otherwise

Since:

23

See Also:

`setStrict(boolean)`,
`parse(String, ParsePosition)`

setStrict

`public void setStrict(boolean strict)`

Change the leniency value for parsing. Parsing can either be strict or lenient, by default it is lenient.

Overrides:

setStrict in class NumberFormat

Parameters:

strict - true if parsing should be done strictly; false otherwise

Since:

23

See Also:

isStrict(),
parse(String, ParsePosition)

isParseBigDecimal

```
public boolean isParseBigDecimal()
```

Returns whether the parse(java.lang.String, java.text.ParsePosition) method returns BigDecimal. The default value is false.

Returns:

true if the parse method returns BigDecimal; false otherwise

Since:

1.5

See Also:

setParseBigDecimal(boolean)

setParseBigDecimal

```
public void setParseBigDecimal(boolean newValue)
```

Sets whether the parse(java.lang.String, java.text.ParsePosition) method returns BigDecimal.

Parameters:

newValue - true if the parse method returns BigDecimal; false otherwise

Since:

1.5

See Also:

isParseBigDecimal()

clone

```
public Object clone()
```

Standard override; no change in semantics.

Overrides:

clone in class NumberFormat

Returns:

a clone of this instance.

See Also:

Cloneable

equals

```
public boolean equals(Object obj)
```

Compares the specified object with this DecimalFormat for equality. Returns true if the object is also a DecimalFormat and the two formats would format any value the same.

Overrides:

equals in class NumberFormat

Implementation Requirements:

This method performs an equality check with a notion of class identity based on getClass(), rather than instanceof. Therefore, in the equals methods in subclasses, no instance of this class should compare as equal to an instance of a subclass.

Parameters:

obj - object to be compared for equality

Returns:

true if the specified object is equal to this DecimalFormat

See Also:

Object.equals(Object)

hashCode

```
public int hashCode()
```

Returns the hash code for this DecimalFormat.

Overrides:

hashCode in class NumberFormat

Implementation Requirements:

This method calculates the hash code value using the values returned from getPositivePrefix() and NumberFormat.hashCode().

Returns:

the hash code for this DecimalFormat

See Also:

`Object.hashCode()`, `NumberFormat.hashCode()`

toString

```
public String toString()
```

Returns a string identifying this `DecimalFormat`, for debugging.

Overrides:

`toString` in class `Object`

Returns:

a string identifying this `DecimalFormat`, for debugging

toPattern

```
public String toPattern()
```

Synthesizes a pattern string that represents the current state of this Format object.

Returns:

a pattern string

See Also:

`applyPattern(String)`

toLocalizedPattern

```
public String toLocalizedPattern()
```

Synthesizes a localized pattern string that represents the current state of this Format object.

Returns:

a localized pattern string

See Also:

`applyPattern(String)`

applyPattern

```
public void applyPattern(String pattern)
```

Apply the given pattern to this Format object. A pattern is a short-hand specification for the various formatting properties. These properties can also be changed individually through the various setter methods.

The number of maximum integer digits is usually not derived from the pattern. See the note in the Patterns section for more detail. For negative numbers, use a second pattern, separated by a semicolon

Example "#,#00.0#" → 1,234.56

This means a minimum of 2 integer digits, 1 fraction digit, and a maximum of 2 fraction digits.

Example: "#,#00.0#;(#,#00.0#)" for negatives in parentheses.

In negative patterns, the minimum and maximum counts are ignored; these are presumed to be set in the positive pattern.

Parameters:

`pattern` - a new pattern

Throws:

`NullPointerException` - if `pattern` is null

`IllegalArgumentException` - if the given pattern is invalid.

applyLocalizedPattern

```
public void applyLocalizedPattern(String pattern)
```

Apply the given pattern to this Format object. The pattern is assumed to be in a localized notation. A pattern is a short-hand specification for the various formatting properties. These properties can also be changed individually through the various setter methods.

The number of maximum integer digits is usually not derived from the pattern. See the note in the Patterns section for more detail. For negative numbers, use a second pattern, separated by a semicolon

Example "#,#00.0#" → 1,234.56

This means a minimum of 2 integer digits, 1 fraction digit, and a maximum of 2 fraction digits.

Example: "#,#00.0#;(#,#00.0#)" for negatives in parentheses.

In negative patterns, the minimum and maximum counts are ignored; these are presumed to be set in the positive pattern.

Parameters:

`pattern` - a new pattern

Throws:

`NullPointerException` - if `pattern` is null

IllegalArgumentException - if the given pattern is invalid.

setMaximumIntegerDigits

```
public void setMaximumIntegerDigits(int newValue)
```

Sets the maximum number of digits allowed in the integer portion of a number during formatting. maximumIntegerDigits must be ≥ minimumIntegerDigits. If the new value for maximumIntegerDigits is less than the current value of minimumIntegerDigits, then minimumIntegerDigits will also be set to the new value. Negative input values are replaced with 0.

Overrides:

setMaximumIntegerDigits in class NumberFormat

Parameters:

newValue - the maximum number of integer digits to be shown.

See Also:

getMaximumIntegerDigits(),
Integer and Fraction Digit Limits

setMinimumIntegerDigits

```
public void setMinimumIntegerDigits(int newValue)
```

Sets the minimum number of digits allowed in the integer portion of a number during formatting. minimumIntegerDigits must be ≤ maximumIntegerDigits. If the new value for minimumIntegerDigits exceeds the current value of maximumIntegerDigits, then maximumIntegerDigits will also be set to the new value. Negative input values are replaced with 0.

Overrides:

setMinimumIntegerDigits in class NumberFormat

Parameters:

newValue - the minimum number of integer digits to be shown.

See Also:

getMinimumIntegerDigits(),
Integer and Fraction Digit Limits

setMaximumFractionDigits

```
public void setMaximumFractionDigits(int newValue)
```

Sets the maximum number of digits allowed in the fraction portion of a number during formatting. maximumFractionDigits must be ≥ minimumFractionDigits. If the new value for maximumFractionDigits is less

than the current value of `minimumFractionDigits`, then `minimumFractionDigits` will also be set to the new value. Negative input values are replaced with 0.

Overrides:

`setMaximumFractionDigits` in class `NumberFormat`

Parameters:

`newValue` - the maximum number of fraction digits to be shown.

See Also:

`getMaximumFractionDigits()`,
Integer and Fraction Digit Limits

setMinimumFractionDigits

```
public void setMinimumFractionDigits(int newValue)
```

Sets the minimum number of digits allowed in the fraction portion of a number during formatting. `minimumFractionDigits` must be ≤ `maximumFractionDigits`. If the new value for `minimumFractionDigits` exceeds the current value of `maximumFractionDigits`, then `maximumFractionDigits` will also be set to the new value. Negative input values are replaced with 0.

Overrides:

`setMinimumFractionDigits` in class `NumberFormat`

Parameters:

`newValue` - the minimum number of fraction digits to be shown.

See Also:

`getMinimumFractionDigits()`,
Integer and Fraction Digit Limits

getMaximumIntegerDigits

```
public int getMaximumIntegerDigits()
```

Returns the maximum number of digits allowed in the integer portion of a number during formatting.

Unlike the other digit limits, `maximumIntegerDigits` is not updated by `DecimalFormats` created or updated with a string pattern.

Overrides:

`getMaximumIntegerDigits` in class `NumberFormat`

Returns:

the maximum number of digits

See Also:

`setMaximumIntegerDigits(int)`,
Pattern Section,
Integer and Fraction Digit Limits

getMinimumIntegerDigits

```
public int getMinimumIntegerDigits()
```

Returns the minimum number of digits allowed in the integer portion of a number during formatting.

Overrides:

`getMinimumIntegerDigits` in class `NumberFormat`

Returns:

the minimum number of digits

See Also:

`setMinimumIntegerDigits(int)`,
Integer and Fraction Digit Limits

getMaximumFractionDigits

```
public int getMaximumFractionDigits()
```

Returns the maximum number of digits allowed in the fraction portion of a number during formatting.

Overrides:

`getMaximumFractionDigits` in class `NumberFormat`

Returns:

the maximum number of digits.

See Also:

`setMaximumFractionDigits(int)`,
Integer and Fraction Digit Limits

getMinimumFractionDigits

```
public int getMinimumFractionDigits()
```

Returns the minimum number of digits allowed in the fraction portion of a number during formatting.

Overrides:

`getMinimumFractionDigits` in class `NumberFormat`

Returns:

the minimum number of digits

See Also:

`setMinimumFractionDigits(int)`,
Integer and Fraction Digit Limits

getCurrency

```
public Currency getCurrency()
```

Gets the currency used by this decimal format when formatting currency values. The currency is obtained by calling `DecimalFormatSymbols.getCurrency` on this number format's symbols.

Overrides:

`getCurrency` in class `NumberFormat`

Returns:

the currency used by this decimal format, or `null`

Since:

1.4

setCurrency

```
public void setCurrency(Currency currency)
```

Sets the currency used by this number format when formatting currency values. This does not update the minimum or maximum number of fraction digits used by the number format. The currency is set by calling `DecimalFormatSymbols.setCurrency` on this number format's symbols.

Overrides:

`setCurrency` in class `NumberFormat`

Parameters:

`currency` - the new currency to be used by this decimal format

Throws:

`NullPointerException` - if `currency` is null

Since:

1.4

getRoundingMode

```
public RoundingMode getRoundingMode()
```

Gets the `RoundingMode` used in this DecimalFormat.

Overrides:

getRoundingMode in class NumberFormat

Returns:

The RoundingMode used for this DecimalFormat.

Since:

1.6

See Also:

setRoundingMode(RoundingMode)

setRoundingMode

```
public void setRoundingMode(RoundingMode roundingMode)
```

Sets the RoundingMode used in this DecimalFormat.

Overrides:

setRoundingMode in class NumberFormat

Parameters:

roundingMode - The RoundingMode to be used

Throws:

NullPointerException - if roundingMode is null.

Since:

1.6

See Also:

getRoundingMode()

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.